AI-Based Electricity Demand Behaviour Intelligence System

Component 1 — Behaviour Intelligence Module Final Year Research Project | Sri Lanka Institute of Information Technology (SLIIT)

Research Information

Student Name	Thanujan T
Student ID	IT22545176
Institution	Sri Lanka Institute of Information Technology (SLIIT)
Supervisor	Dr. Nathali Silva
Research Component	Component 1 — Electricity Demand Behaviour Intelligence
Academic Year	2025 / 2026

Abstract

This research component develops an AI-based electricity demand behaviour intelligence system that analyses historical electricity consumption data to identify, classify, and forecast demand behaviour patterns. The system applies unsupervised machine learning techniques — K-Means clustering, DBSCAN, and Isolation Forest anomaly detection — to extract 96-slot daily load profiles from raw time-series data and categorise each day into one of four behaviour archetypes: Normal Weekday Demand, Peak Demand Day, Abnormal Demand Day, and Holiday / Low Demand Day.

The classified patterns are exposed through a RESTful API and visualised on an interactive React dashboard, enabling utility providers and energy managers to monitor consumption trends, detect anomalous demand events, and predict future behaviour using seasonal historical matching. The system also integrates LIME-based explainability to provide interpretable justifications for abnormal day classifications.

Research Problem & Motivation

Electricity demand forecasting and behaviour classification remain challenging due to the high variability and nonlinearity of consumption patterns across time. Traditional threshold-based approaches fail to capture complex daily load shape dynamics, particularly for peak detection and anomaly identification. This component addresses the gap by combining clustering, anomaly detection, and rule-based classification into a unified pipeline that:

- Automatically learns daily load profile shapes without labelled training data
- Detects abnormal demand events using a multi-method ensemble approach
- Provides human-interpretable explanations for AI-driven classifications
- Supports real-time querying and future behaviour forecasting

Research Objectives

1. Extract and normalise 96-slot daily demand profiles from raw 15-minute interval electricity data
2. Cluster daily profiles using K-Means and DBSCAN to discover natural demand archetypes
3. Detect anomalous demand days using Isolation Forest and Z-score ensemble methods
4. Apply a priority-based rule engine to assign interpretable behaviour labels to every day
5. Forecast future behaviour using seasonal matching on historical classified data
6. Deliver results through a RESTful API and an interactive analytics dashboard with XAI support

---

System Architecture



Technology Stack

Backend

Technology	Version	Role
Python	3.10+	Core language

FastAPI	0.111.0	REST API framework
Uvicorn	0.29.0	ASGI server
scikit-learn	1.4.2	K-Means, DBSCAN, Isolation Forest, StandardScaler
pandas	2.2.2	Time-series data processing
numpy	1.26.4	Numerical computation
scipy	1.13.0	Statistical functions
Pydantic	2.7.1	Request / response validation

Frontend

Technology	Version	Role
React	18.3.1	UI component framework
Vite	5.0.0	Build tool and dev server
Recharts	2.12.7	Interactive data visualisation
Axios	1.7.2	HTTP client for API communication

Project Structure

```
behaviour-intelligence/
│
├── backend/
│   └── app/
│       ├── main.py                        # FastAPI application entry point & CORS
│       ├── api/
│       │   └── routes.py                  # All HTTP endpoint definitions
│       ├── controllers/
│       │   └── pipeline_controller.py     # Orchestrates ML pipeline steps
│       ├── core/
│       │   └── config.py                  # Project-wide path and URL constants
│       ├── ml/
│       │   ├── train_models.py            # Standalone 8-step training script (entry point)
│       │   ├── pipeline_runner.py         # Callable pipeline executor function
│       │   ├── data_loader.py             # CSV loader with auto column detection
│       │   ├── preprocessing.py           # Cleaning, gap-filling, 15-min resampling
│       │   ├── daily_profile.py           # 96-slot daily load profile generation
│       │   ├── clustering.py              # K-Means (auto-k) & DBSCAN training
│       │   ├── anomaly_detection.py       # Isolation Forest & Z-score ensemble
│       │   ├── peak_detection.py          # Percentile-based peak day identification
│       │   ├── behavior_labeling.py       # Priority-rule behaviour classification
│       │   ├── behavior_prediction.py     # Seasonal matching future forecasting
│       │   └── verify_pkl_models.py       # Trained model file verification
│       ├── schemas/
│       │   └── response_models.py         # Pydantic response type definitions
│       └── services/
│           └── analysis_service.py        # Dashboard KPI and profile queries
│
├── frontend/
│   ├── src/
│   │   ├── App.jsx                        # Main dashboard component
│   │   ├── App.css                        # Dashboard stylesheet (dark theme)
│   │   ├── FutureBehaviorPrediction.jsx   # Prediction panel UI component
│   │   └── main.jsx                       # Vite application entry point
│   ├── public/
```

```
|   |   └── index.html
|   ├── package.json
|   └── vite.config.js
|
├── data/
|   ├── raw/
|   |   └── load_forecasting_dataset_corrected.csv   # Raw input dataset
|   └── processed/                          # Auto-generated by training pipeline
|       ├── cleaned_data.csv                # Cleaned 15-min time series
|       ├── daily_profiles.csv              # 96-slot profiles per day
|       ├── clustering_results.csv          # Cluster assignments
|       ├── anomaly_results.csv             # Anomaly flags and scores
|       ├── peak_demand_results.csv         # Peak day flags
|       └── behavior_labels.csv             # Final behaviour classification per day
|
├── models/                                 # Auto-generated by training pipeline
|   ├── profile_scaler.pkl                  # StandardScaler for 96-point profiles
|   ├── kmeans_model.pkl                    # Trained K-Means model
|   ├── dbscan_model.pkl                    # Trained DBSCAN model
|   ├── isolation_forest_model.pkl          # Trained Isolation Forest model
|   ├── clustering_metrics.json             # Silhouette and cluster statistics
|   ├── anomaly_metrics.json                # Anomaly detection statistics
|   └── behavior_label_summary.json         # Label distribution summary
|
├── notebooks/
|   ├── 01_data_preprocessing.ipynb
|   ├── 02_daily_profile_generation.ipynb
|   ├── 03_clustering_model_training.ipynb
|   ├── 04_anomaly_detection_training.ipynb
|   └── 05_behavior_label_generation.ipynb
|
├── reports/
|   ├── training_pipeline_summary.md
|   └── model_training_outputs.md
|
├── requirements.txt
└── README.md
```

Setup & Installation

Prerequisites

- Python 3.10 or higher
- Node.js 18 or higher
- npm 9 or higher

---

1. Clone the Repository

```
git clone <repository-url>
cd behaviour-intelligence
```

2. Install Backend Dependencies

```
pip install -r requirements.txt
```

3. Verify Raw Dataset

Ensure the input dataset is present at:

```
data/raw/load_forecasting_dataset_corrected.csv
```

The file must contain a **timestamp column** and an **electricity demand column**. The loader detects column names automatically.

4. Run the ML Training Pipeline

Execute the full 8-step training pipeline from the **project root**:

```
python backend/app/ml/train_models.py
```

This will: - Process and clean the raw data - Generate 96-slot daily load profiles - Train and save all ML models to `models/` - Generate all processed CSVs to `data/processed/` - Output training metrics and reports

> **Important:** Always run from the `behaviour-intelligence/` root directory.

5. Start the Backend API

```
python -m uvicorn backend.app.main:app --reload --port 8001
```

6. Install and Start the Frontend

```
cd frontend
npm install
npm start
```

7. Access the System

Service	URL
React Dashboard	http://localhost:3000
API Base URL	http://127.0.0.1:8001
Swagger API Docs	http://127.0.0.1:8001/docs
Health Check	http://127.0.0.1:8001/health

ML Pipeline — Detailed Steps

Step	Module	Method	Description
1	`data_loader.py`	Auto-detection	Load CSV; detect timestamp and demand columns automatically
2	`preprocessing.py`	IQR + interpolation	Remove NaN, negatives, and 3×IQR outlier spikes; linear interpolation for gaps; resample to 15-min intervals
3	`daily_profile.py`	Pivot aggregation	Convert 15-min time series into 96-slot vectors (one per day); compute peak, mean, std per day
4	`clustering.py`	K-Means + DBSCAN	Normalise profiles with StandardScaler; select optimal k via silhouette score; fit both models
5	`anomaly_detection.py`	Isolation Forest + Z-score	Contamination = 3%; Z-score threshold = 3.0; combine flags with OR logic

6	`peak_detection.py`	Percentile thresholds	Flag peak days: 90th percentile peak OR z-score > 2.0 OR (anomaly + 75th percentile)
7	`behavior_labeling.py`	Priority-rule engine	Apply ordered rules to assign final label and risk level
8	Evaluation	Metrics export	Silhouette scores, anomaly statistics, label distribution; saved as JSON

Behaviour Classification System

Each day receives exactly one label based on a **priority-ordered rule engine**:

Priority	Label	Risk Level	Trigger Condition
1 (Highest)	**Abnormal Demand Day**	High	Anomaly flag = True AND peak flag = False
2	**Peak Demand Day**	Medium	Peak ≥ 90th percentile OR peak z-score > 2.0 OR (anomaly + peak ≥ 75th percentile)
3	**Holiday / Low Demand Day**	Low	Mean daily demand < 25th percentile
4 (Default)	**Normal Weekday Demand**	Normal	All other days

Future Behaviour Prediction

The prediction module (`behavior_prediction.py`) forecasts the expected label for any future target date using seasonal matching:

1. **Primary match** — same month + same day-of-week from historical labelled data
2. **Fallback 1** — same month only (all day-of-week values)
3. **Fallback 2** — full historical dataset

The most frequent label among matched days is returned as the prediction with a probability distribution.

API Reference

All endpoints are prefixed with `/api`. Backend runs on port **8001**.

Method	Endpoint	Description
`GET`	`/health`	System health check
`POST`	`/api/generate-daily-profiles`	Preprocess raw data; generate 96-slot daily profiles
`POST`	`/api/run-clustering`	Train K-Means and DBSCAN on daily profiles
`POST`	`/api/run-anomaly-detection`	Run Isolation Forest + Z-score anomaly detection
`POST`	`/api/detect-peak-days`	Identify peak demand days via percentile thresholds
`POST`	`/api/generate-behavior-labels`	Assign behaviour labels to all days

GET	/api/dashboard-summary	Full KPI summary for the dashboard
GET	/api/predict-future-behavior?date=YYYY-MM-DD	Predict behaviour label for a given future date
GET	/api/get-day-profile?date=YYYY-MM-DD	Retrieve the 96-slot load profile for a specific date

Interactive API documentation (Swagger UI): **http://127.0.0.1:8001/docs**

Dashboard Features

The React frontend delivers a full interactive analytics dashboard:

Feature	Description
KPI Summary Cards	Counts of Total, Normal, Peak, Abnormal, and Holiday days
Label Distribution Pie Chart	Clustering-based behaviour breakdown
Risk Distribution Bar Chart	Days grouped by risk level
Top 15 Anomalous Days	Bar chart of highest anomaly-scored days
Anomaly Score Trend	180-day rolling line chart of anomaly scores
All Days Explorer	Paginated table of all classified days with detail panel
96-Point Load Profile Viewer	Full 24-hour demand curve for any selected date
Future Behaviour Prediction	Date picker with seasonal-matching prediction output
XAI / LIME Explainer	Interpretable feature importance for abnormal day classifications

---

Model Verification

To verify all trained model files are correctly saved and loadable after training:

```
python backend/app/ml/verify_pkl_models.py
```

Expected models: `profile_scaler.pkl`, `kmeans_model.pkl`, `dbscan_model.pkl`, `isolation_forest_model.pkl`

Notebooks

Step-by-step Jupyter notebooks are provided for each pipeline stage:

Notebook	Pipeline Step
`01_data_preprocessing.ipynb`	Data loading, cleaning, resampling
`02_daily_profile_generation.ipynb`	96-slot profile construction
`03_clustering_model_training.ipynb`	K-Means and DBSCAN training
`04_anomaly_detection_training.ipynb`	Isolation Forest and Z-score detection
`05_behavior_label_generation.ipynb`	Rule-based behaviour labeling

Additional Documentation

- `reports/training_pipeline_summary.md` — Complete training guide with expected outputs
- `reports/model_training_outputs.md` — Detailed model metrics and results

License

This project is developed for academic research purposes at SLIIT. All rights reserved.